

AI-Assisted Coding Lab Assignment=11.1

Name :Pothunuri Akshitha

Roll no: 2403A510C2

Batch no:05

CSE 2nd year

Task Description #1 – Stack Implementation

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

Sample Input Code:

```
class Stack:
```

```
pass
```

Prompt:

Generate a Stack class with push, pop, peek, and is_empty methods. Sample input should be like class Stack:pass

Sample output should be like A functional stack implementation with all required methods and docstrings along with print statements and it should take input from user and then give output as expected .

Code Generated:

```

11.1.task1.py > ...
1  class Stack:
2      """
3      A simple LIFO (last-in, first-out) stack implementation.
4
5      Methods
6      -----
7      push(item):
8          Pushes an item onto the top of the stack and prints the action.
9
10     pop():
11         Removes and returns the top item from the stack. Prints the action
12         and handles underflow by printing a message and returning None.
13
14     peek():
15         Returns the top item without removing it. Prints the action and
16         returns None if the stack is empty.
17
18     is_empty():
19         Returns True if the stack has no items; otherwise False. Also prints
20         the current empty state.
21     """
22
23     def __init__(self):
24         """Initialize an empty stack."""
25         self._items = []
26         print("Initialized an empty stack.")
27
28     def push(self, item):

```

```

28     def push(self, item):
29         """Push an item onto the top of the stack.
30
31         Parameters
32         -----
33         item : Any
34             The item to push onto the stack.
35         """
36         self._items.append(item)
37         print(f"Pushed: {item}. Current stack: {self._items}")
38
39     def pop(self):
40         """Remove and return the top item from the stack.
41
42         Returns
43         -----
44         Any | None
45             The popped item, or None if the stack is empty.
46         """
47         if self.is_empty():
48             print("Pop attempted on empty stack. Nothing to pop.")
49             return None
50         item = self._items.pop()
51         print(f"Popped: {item}. Current stack: {self._items}")
52         return item
53
54     def peek(self):

```

```

11.1.task1.py > ...
1  class Stack:
54     def peek(self):
55         """Return the top item from the stack without removing it.
56
57         Returns
58         -----
59         Any | None
60         | The top item, or None if the stack is empty.
61         """
62         if self.is_empty():
63             print("Peek on empty stack. Top item: None")
64             return None
65         top_item = self._items[-1]
66         print(f"Peeked: {top_item}. Current stack: {self._items}")
67         return top_item
68
69     def is_empty(self):
70         """Return whether the stack is empty.
71
72         Returns
73         -----
74         bool
75         | True if the stack is empty, else False.
76         """
77         empty = len(self._items) == 0
78         print(f"Stack is empty: {empty}")
79         return empty
80
81
82     if __name__ == "__main__":
83         stack = Stack()
84         stack.push(10)
85         stack.push(20)
86         stack.peek()
87         stack.pop()
88         stack.is_empty()

```

Output:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  Filter
[Running] python -u "c:\Users\akshi\OneDrive\Desktop\AI_Assisted_Coding\11.1.task1.py"
Initialized an empty stack.
Pushed: 10. Current stack: [10]
Pushed: 20. Current stack: [10, 20]
Stack is empty: False
Peeked: 20. Current stack: [10, 20]
Stack is empty: False
Popped: 20. Current stack: [10]
Stack is empty: False

[Done] exited with code=0 in 0.095 seconds

```

Observation:

Implemented a user-interactive LIFO Stack with push, pop, peek, and is_empty, including clear docstrings and state prints.

CLI exits on typing "pass", ensuring simple, guided input and visible stack state after each action.

Task Description #2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.

Sample Input Code:

```
class Queue:
```

```
pass
```

Prompt:

Use AI to implement a Queue using Python lists.

Sample Input Code should be like

class Queue:pass. User should give the input and then we have to get output as FIFO-based queue class with enqueue, dequeue, peek, and size methods.

Code Generated:

```
11.1.task2.py > ...
1  class Queue:
2      """A simple FIFO queue implemented using a Python list.
3
4      Methods
5      -----
6      enqueue(item): Add an item to the back of the queue.
7      dequeue(): Remove and return the front item (raises IndexError if empty).
8      peek(): Return the front item without removing it (raises IndexError if empty).
9      size(): Return the number of items in the queue.
10     is_empty(): Return True if the queue has no items.
11     """
12
13     def __init__(self):
14         self._items = []
15
16     def enqueue(self, item):
17         """Add an item to the back of the queue."""
18         self._items.append(item)
19
20     def dequeue(self):
21         """Remove and return the front item. Raises IndexError if empty."""
22         if not self._items:
23             raise IndexError("dequeue from empty queue")
24         return self._items.pop(0)
25
26     def peek(self):
27         """Return the front item without removing it. Raises IndexError if empty."""
28         if not self._items:
29             raise IndexError("peek from empty queue")
30         return self._items[0]
31
32     def size(self):
33         """Return the number of items in the queue."""
34         return len(self._items)
35
36     def is_empty(self):
37         """Return True if the queue has no items."""
```

```

38         return not self._items
39
40     def __len__(self):
41         return self.size()
42
43     def __repr__(self):
44         return f"Queue({self._items!r})"
45
46
47     def _print_menu():
48         print("\nChoose an operation:")
49         print("1. Enqueue")
50         print("2. Dequeue")
51         print("3. Peek")
52         print("4. Size")
53         print("5. Is Empty")
54         print("6. Show Queue")
55         print("0. Exit")
56
57
58 if __name__ == "__main__":
59     q = Queue()
60     print("FIFO Queue Demo (enqueue, dequeue, peek, size).")
61     while True:
62         _print_menu()
63         choice = input("Enter choice: ").strip()
64
65         if choice == "1":
66             value = input("Enter value to enqueue: ")
67             q.enqueue(value)
68             print(f"Enqueued: {value}")
69         elif choice == "2":
70             try:
71                 value = q.dequeue()
72                 print(f"Dequeued: {value}")
73             except IndexError as e:
74                 print(f"Error: {e}")
75         elif choice == "3":
76             try:
77                 print(f"Front: {q.peek()}")
78             except IndexError as e:
79                 print(f"Error: {e}")
80         elif choice == "4":
81             print(f"Size: {q.size()}")
82         elif choice == "5":
83             print(f"Is Empty: {q.is_empty()}")
84         elif choice == "6":
85             print(q)
86         elif choice == "0":
87             print("Exiting.")
88             break
89         else:
90             print("Invalid choice. Please try again.")

```

Output:

```
[Running] python -u "c:\Users\akshi\OneDrive\Desktop\AI_Assisted_Coding\11.1.task2.py"  
FIFO Queue Demo (enqueue, dequeue, peek, size).
```

```
Choose an operation:
```

1. Enqueue
 2. Dequeue
 3. Peek
 4. Size
 5. Is Empty
 6. Show Queue
 0. Exit
- ```
Enter choice: |
```

### Observation:

Implemented a FIFO Queue with enqueue, dequeue, peek, and size, plus a simple CLI for user input.

Uses a Python list with pop(0) (fine for small queues); for large workloads, prefer collections.deque for O(1) front pops.

### Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods.

Sample Input Code:

```
class Node:
```

```
pass
```

### Prompt:

Use AI to generate a Singly Linked List with insert and display methods. user should give

Sample Input Code should be like class Node:pass class Linked: List

Expected Output:

- A working linked list implementation with clear method documentation.

### Code generated:

```

11.1.task3.py > ...
1 class Node:
2 """A node in a singly linked list.
3
4 Attributes:
5 data: The value stored in the node.
6 next: Reference to the next node in the list, or None.
7 """
8
9 def __init__(self, data): # Fixed __init__
10 self.data = data
11 self.next = None
12
13
14 class LinkedList:
15 """A simple singly linked list with insert and display operations."""
16
17 def __init__(self): # Fixed __init__
18 self.head = None
19
20 def insert(self, value):
21 """Insert a new node with the given value at the end of the list.
22
23 Args:
24 value: The value to insert into the list.
25 """
26 new_node = Node(value)
27 if self.head is None:
28 self.head = new_node
29 return
30 current = self.head
31 while current.next is not None:
32 current = current.next
33 current.next = new_node
34
35 def display(self):
36 """Return a string representation of the list's elements.
37
38 Returns:
39 A string with elements joined by ' -> '. If the list is empty, returns 'Empty'.
40 """
41 if self.head is None:
42 return "Empty"
43 values = []
44 current = self.head
45 while current is not None:
46 values.append(str(current.data))
47 current = current.next
48 return " -> ".join(values)
49
50
51 if __name__ == "__main__": # Fixed __name__
52 # Sample input: user enters values separated by spaces
53 # Example: 10 20 30 40
54 user_input = input("Enter values to insert (space-separated): ").strip()
55
56 linked_list = LinkedList()
57 if user_input:
58 for token in user_input.split():
59 # Try to convert to int if possible; otherwise keep as string
60 try:
61 value = int(token)
62 except ValueError:
63 try:
64 value = float(token)
65 except ValueError:
66 value = token
67 linked_list.insert(value)
68
69 print("Linked List:")
70 print(linked_list.display())
71

```

**Output:**

```
[Done] exited with code=1 in 0.245 seconds
```

```
[Running] python -u "c:\Users\akshi\OneDrive\Desktop\AI_Assisted_Coding\11.1.task3.py"
Enter values to insert (space-separated): |
```

### Observation:

Implemented a singly linked list (Node, LinkedList) with insert (append) and display, including clear docstrings and a simple CLI for user input.

Handles empty list gracefully and parses numbers/strings; insert is  $O(n)$  due to tail traversal.

### Task Description #4 – Binary Search Tree (BST)

Task: Use AI to create a BST with insert and in-order traversal methods.

Sample Input Code:

```
class BST:
```

```
pass
```

### Prompt:

create a BST with insert and in-order traversal methods.

Sample Input Code:

```
class BST:
```

```
pass
```

### Code Generated:



11.1.task4.py > ...

```
1 class BSTNode:
2 """
3 Node class for Binary Search Tree.
4
5 Args:
6 value (int): The value stored in the node.
7 left (BSTNode): Left child node.
8 right (BSTNode): Right child node.
9 """
10 def __init__(self, value):
11 self.value = value
12 self.left = None
13 self.right = None
14
15 class BST:
16 """
17 Binary Search Tree implementation with insert and in-order traversal methods.
18 """
19 def __init__(self):
20 self.root = None
21
22 def insert(self, value):
23 """
24 Inserts a value into the BST.
25
26 Args:
27 value (int): The value to insert.
28 """
29 if self.root is None:
30 self.root = BSTNode(value)
31 else:
32 self._insert(self.root, value)
33
34 def _insert(self, node, value):
35 if value < node.value:
36 if node.left is None:
```

```

37 node.left = BSTNode(value)
38 else:
39 self._insert(node.left, value)
40 else:
41 if node.right is None:
42 node.right = BSTNode(value)
43 else:
44 self._insert(node.right, value)
45
46 def in_order_traversal(self):
47 """
48 Performs in-order traversal of the BST.
49
50 Returns:
51 | list: List of values in in-order.
52 """
53 result = []
54 self._in_order(self.root, result)
55 return result
56
57 def _in_order(self, node, result):
58 if node:
59 self._in_order(node.left, result)
60 result.append(node.value)
61 self._in_order(node.right, result)
62
63 # Example usage
64 bst = BST()
65 for num in [7, 3, 9, 1, 5]:
66 bst.insert(num)
67 print(bst.in_order_traversal()) # Output: [1, 3, 5, 7, 9]

```

## Output:

```

[Running] python -u "c:\Users\akshi\OneDrive\Desktop\AI_Assisted_Coding\11.1.task4.py"
[1, 3, 5, 7, 9]

[Done] exited with code=0 in 0.124 seconds

```

## Observation:

1. The code defines a **Binary Search Tree (BST)** with nodes that store values and pointers to left and right children.
2. The insert method correctly places values in the BST while maintaining the **BST property** (left < root < right).
3. The in\_order\_traversal method returns the values in **sorted ascending order**.
4. For the input [7, 3, 9, 1, 5], the output is [1, 3, 5, 7, 9], confirming correct functionality.

## Task Description #5 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and delete methods.

Sample Input Code:

```
class HashTable:
```

```
pass
```

### Prompt:

Implement a hash table with basic insert, search, and delete methods.

Sample Input Code:

```
class HashTable:
```

```
pass
```

### Code Generated:

```
11.1.task5.py > ...
1 class HashTable:
2 """
3 A simple hash table implementation using separate chaining for collision resolution.
4
5 Methods
6 -----
7 insert(key, value):
8 Inserts a key-value pair into the hash table.
9
10 search(key):
11 Searches for a value by key and returns it if found, else returns None.
12
13 delete(key):
14 Deletes a key-value pair from the hash table if the key exists.
15 """
16
17 def __init__(self, size=10):
18 """Initialize the hash table with a fixed size."""
19 self.size = size
20 self.table = [[] for _ in range(size)]
21
22 def _hash(self, key):
23 """Compute the hash index for a given key."""
24 return hash(key) % self.size
25
26 def insert(self, key, value):
27 """Insert a key-value pair into the hash table."""
28 index = self._hash(key)
29 # Check if key exists and update
30 for i, (k, v) in enumerate(self.table[index]):
31 if k == key:
```

```

20 def insert(self, key, value):
21 # Check if key exists
22 index = self._hash(key)
23 self.table[index][i] = (key, value)
24 print(f"Updated key '{key}' with value '{value}'.")
25 return
26 # Otherwise, insert new
27 self.table[index].append((key, value))
28 print(f"Inserted key '{key}' with value '{value}'.")
29
30 def search(self, key):
31 """Search for a value by key in the hash table."""
32 index = self._hash(key)
33 for k, v in self.table[index]:
34 if k == key:
35 print(f"Found key '{key}' with value '{v}'.")
36 return v
37 print(f"Key '{key}' not found.")
38 return None
39
40 def delete(self, key):
41 """Delete a key-value pair from the hash table."""
42 index = self._hash(key)
43 for i, (k, v) in enumerate(self.table[index]):
44 if k == key:
45 del self.table[index][i]
46 print(f"Deleted key '{key}'.")
47 return True
48 print(f"Key '{key}' not found for deletion.")
49 return False
50
51 # Example usage
52 if __name__ == "__main__":
53 ht = HashTable()
54 ht.insert("apple", 100)
55 ht.insert("banana", 200)
56 ht.search("apple")
57 ht.delete("banana")
58 ht.search("banana")

```

## Output:

```

[Running] python -u "c:\Users\akshi\OneDrive\Desktop\AI_Assisted_Coding\11.1.task5.py"
Inserted key 'apple' with value '100'.
Inserted key 'banana' with value '200'.
Found key 'apple' with value '100'.
Deleted key 'banana'.
Key 'banana' not found.

[Done] exited with code=0 in 0.119 seconds

```

## Observation:

- ☐ The code implements a Hash Table using separate chaining (lists at each index) to handle collisions.
- ☐ It provides methods to insert, search, and delete key-value pairs efficiently.
- ☐ Keys are hashed using Python's built-in hash() function and mapped within the fixed table size.
- ☐ Example usage shows correct behavior: inserting keys, updating, finding values, and handling deletion properly.

## Task Description #6 – Graph Representation

Task: Use AI to implement a graph using an adjacency list.

Sample Input Code:

```
class Graph:
```

```
pass
```

**Prompt:**

Implement a graph using an adjacency list Sample Input Code:

```
class Graph:
```

```
pass
```

**Code Generated:**

```
11.1.task6.py > ...
1 class Graph:
2 """
3 Graph implementation using an adjacency list.
4
5 Methods
6 -----
7 add_vertex(vertex):
8 | Adds a vertex to the graph.
9
10 add_edge(vertex1, vertex2):
11 | Adds an edge between vertex1 and vertex2.
12
13 get_adjacent(vertex):
14 | Returns a list of adjacent vertices for the given vertex.
15 """
16
17 def __init__(self):
18 """Initialize an empty adjacency list."""
19 self.adj_list = {}
20
21 def add_vertex(self, vertex):
22 """Add a vertex to the graph."""
23 if vertex not in self.adj_list:
24 self.adj_list[vertex] = []
25
26 def add_edge(self, vertex1, vertex2):
27 """Add an edge between vertex1 and vertex2."""
28 if vertex1 not in self.adj_list:
29 self.add_vertex(vertex1)
30 if vertex2 not in self.adj_list:
31 self.add_vertex(vertex2)
```

```

31 self.add_vertex(vertex2)
32 self.adj_list[vertex1].append(vertex2)
33 self.adj_list[vertex2].append(vertex1) # For undirected graph
34
35 def get_adjacent(self, vertex):
36 """Return a list of adjacent vertices for the given vertex."""
37 return self.adj_list.get(vertex, [])
38
39 # Example usage
40 if __name__ == "__main__":
41 graph = Graph()
42 graph.add_vertex("A")
43 graph.add_vertex("B")
44 graph.add_edge("A", "B")
45 graph.add_edge("A", "C")
46 print("Adjacency List:", graph.adj_list)
47 print("Adjacent to A:", graph.get_adjacent("A"))

```

## Output:

```

[Done] exited with code=0 in 0.119 seconds

[Running] python -u "c:\Users\akshi\OneDrive\Desktop\AI_Assisted_Coding\11.1.task6.py"
Adjacency List: {'A': ['B', 'C'], 'B': ['A'], 'C': ['A']}
Adjacent to A: ['B', 'C']

[Done] exited with code=0 in 0.161 seconds

```

## Observation:

- ☐ The code defines a **Graph class** using an adjacency list representation.
- ☐ Vertices can be added explicitly, and edges automatically add missing vertices.
- ☐ The graph is **undirected**, so edges are stored in both directions.
- ☐ Example usage builds a small graph and shows adjacency like {'A': ['B', 'C'], 'B': ['A'], 'C': ['A']}.

## Task Description #7 – Priority Queue

Task: Use AI to implement a priority queue using Python's heapq module.

Sample Input Code:

```

class PriorityQueue:
 pass

```

## Prompt:

Implement a PriorityQueue class in Python using the built-in heapq module. Your class should support the following methods:

- `push(item, priority)`: Add an item with the given priority to the queue.
- `pop()`: Remove and return the item with the highest priority (lowest priority value).
- `peek()`: Return the item with the highest priority without removing it.
- `is_empty()`: Return True if the queue is empty, otherwise False.

#### Code Generated:

```
11.1.task7.py > PriorityQueue > is_empty
1 import heapq
2
3 class PriorityQueue:
4 def __init__(self):
5 self._heap = []
6
7 def push(self, item, priority):
8 heapq.heappush(self._heap, (priority, item))
9
10 def pop(self):
11 if self.is_empty():
12 raise IndexError("pop from empty priority queue")
13 return heapq.heappop(self._heap)[1]
14
15 def peek(self):
16 if self.is_empty():
17 raise IndexError("peek from empty priority queue")
18 return self._heap[0][1]
19
20 def is_empty(self):
21 return len(self._heap) == 0
22
23 if __name__ == "__main__":
24 pq = PriorityQueue()
25 pq.push('task1', 2)
26 pq.push('task2', 1)
27 print(pq.pop()) # Output: 'task2'
28 print(pq.peek()) # Output: 'task1'
29 print(pq.is_empty()) # Output: False
```

#### Output:

```
[Running] python -u "c:\Users\akshi\OneDrive\Desktop\AI_Assisted_Coding\11.1.task7.py"
task2
task1
False

[Done] exited with code=0 in 0.159 seconds
```

### Observation:

- The code defines a PriorityQueue class using Python's heapq module to maintain a min-heap.
- Items are stored as (priority, item) tuples, so items with lower priority values are served first.
- The push method adds items with their priority to the queue.
- The pop method removes and returns the item with the highest priority (lowest value).
- The peek method returns the highest priority item without removing it.
- The is\_empty method checks if the queue is empty.
- In the sample usage, 'task1' (priority 2) and 'task2' (priority 1) are added.
  - o pop() returns 'task2' (since priority 1 < 2).
  - o peek() returns 'task1'.
  - o is\_empty() returns False because one item remains.
- The code raises IndexError if pop or peek is called on an empty queue.

### Task Description #8 – Deque

Task: Use AI to implement a double-ended queue using collections.deque.

Sample Input Code:

```
class DequeDS:
 pass
```

#### Prompt:

Implement a DequeDS class in Python using the built-in collections.deque module. Your class should support the following methods:

- add\_front(item): Add an item to the front of the deque.
- add\_rear(item): Add an item to the rear of the deque.
- remove\_front(): Remove and return the item from the front.
- remove\_rear(): Remove and return the item from the rear.
- peek\_front(): Return the item at the front without removing it.
- peek\_rear(): Return the item at the rear without removing it.
- is\_empty(): Return True if the deque is empty, otherwise False.

#### Code Generated:



```

11.1.task8.py > ...
1 from collections import deque
2
3 class DequeDS:
4 def __init__(self): # Fixed __init__
5 self._deque = deque()
6
7 def add_front(self, item):
8 self._deque.appendleft(item)
9
10 def add_rear(self, item):
11 self._deque.append(item)
12
13 def remove_front(self):
14 if self.is_empty():
15 raise IndexError("remove_front from empty deque")
16 return self._deque.popleft()
17
18 def remove_rear(self):
19 if self.is_empty():
20 raise IndexError("remove_rear from empty deque")
21 return self._deque.pop()
22
23 def peek_front(self):
24 if self.is_empty():
25 raise IndexError("peek_front from empty deque")
26 return self._deque[0]
27
28 def peek_rear(self):
29 if self.is_empty():
30 raise IndexError("peek_rear from empty deque")
31 return self._deque[-1]
32
33 def is_empty(self):
34 return len(self._deque) == 0
35
36
37 if __name__ == "__main__": # Fixed __name__
38 dq = DequeDS()
39 dq.add_front(1)
40 dq.add_rear(2)
41 print(dq.remove_front()) # Output: 1
42 print(dq.peek_rear()) # Output: 2
43 print(dq.is_empty()) # Output: False
44

```

## Output:

```

[Running] python -u "c:\Users\akshi\OneDrive\Desktop\AI_Assisted_Coding\11.1.task8.py"
1
2
False

[Done] exited with code=0 in 0.158 seconds

```

## Observation:

- The code implements a double-ended queue (DequeDS) using Python's collections.deque.
- Items can be added or removed from both the front and rear efficiently.
- The class provides methods to add, remove, and peek at both ends, as well as check if the deque is empty.

- In the sample usage:
  - o `add_front(1)` adds 1 to the front.
  - o `add_rear(2)` adds 2 to the rear.
  - o `remove_front()` removes and prints 1.
  - o `peek_rear()` prints 2 (the only remaining item).
  - o `is_empty()` prints False since one item remains.
- The code raises `IndexError` if removal or peek operations are attempted on an empty deque.

### Task Description #9 – AI-Generated Data Structure Comparisons

Task: Use AI to generate a comparison table of different data structures (stack, queue, linked list, etc.) including time complexities.

Sample Input Code:

# No code, prompt AI for a data structure comparison table

#### Prompt:

Generate a comparison table of different data structures (such as stack, queue, linked list, array, hash table, binary search tree, heap, etc.) including their typical time complexities for common operations like insertion, deletion, access/search, and update. Present the table in Markdown format.

#### Code Generated:

```
11.1.task9.py > ...
1 def print_data_structure_table():
2 table = [
3 ["Data Structure", "Insertion", "Deletion", "Access/Search", "Update"],
4 ["Array", "O(1)", "O(n)", "O(1)", "O(1)"],
5 ["Stack (Array)", "O(1)", "O(1)", "O(n)", "O(n)"],
6 ["Queue (Array)", "O(1)", "O(1)", "O(n)", "O(n)"],
7 ["Singly Linked List", "O(1)", "O(1)", "O(n)", "O(n)"],
8 ["Doubly Linked List", "O(1)", "O(1)", "O(n)", "O(n)"],
9 ["Hash Table", "O(1)", "O(1)", "O(1)", "O(1)"],
10 ["Binary Search Tree*", "O(log n)", "O(log n)", "O(log n)", "O(log n)"],
11 ["Heap", "O(log n)", "O(log n)", "O(n)", "O(log n)"]
12]
13 for row in table:
14 print("{:<20} {:<10} {:<10} {:<15} {:<10}".format(*row))
15 print("\n*For balanced BSTs (e.g., AVL, Red-Black Tree); unbalanced BSTs may degrade to O(n).")
16
17
18 if __name__ == "__main__": # Fixed __name__
19 print_data_structure_table()
20 from collections import deque
```

#### Output:

```
[Done] exited with code=1 in 0.139 seconds

[Running] python -u "c:\Users\akshi\OneDrive\Desktop\AI_Assisted_Coding\11.1.task9.py"
Data Structure Insertion Deletion Access/Search Update
Array O(1) O(n) O(1) O(1)
Stack (Array) O(1) O(1) O(n) O(n)
Queue (Array) O(1) O(1) O(n) O(n)
Singly Linked List O(1) O(1) O(n) O(n)
Doubly Linked List O(1) O(1) O(n) O(n)
Hash Table O(1) O(1) O(1) O(1)
Binary Search Tree* O(log n) O(log n) O(log n) O(log n)
Heap O(log n) O(log n) O(n) O(log n)

*For balanced BSTs (e.g., AVL, Red-Black Tree); unbalanced BSTs may degrade to O(n).

[Done] exited with code=0 in 0.136 seconds
```

### Observation:

The code defines a function `print_data_structure_table()` that prints a formatted comparison table of common data structures and their time complexities for insertion, deletion, access/search, and update operations.

- The table includes: Array, Stack (Array), Queue (Array), Singly Linked List, Doubly Linked List, Hash Table, Binary Search Tree (BST), and Heap.
- Each row shows the typical time complexity for each operation.
- A note clarifies that BST complexities assume balanced trees; unbalanced BSTs may degrade to  $O(n)$ .
- When run, the code displays the table in a readable, aligned format in the console, making it easy to compare data structures at a glance.

### Task Description #10 Real-Time Application Challenge – Choose the Right Data Structure

Scenario:

Your college wants to develop a Campus Resource Management System that handles:

1. Student Attendance Tracking – Daily log of students entering/exiting the campus.
2. Event Registration System – Manage participants in events with quick search and removal.
3. Library Book Borrowing – Keep track of available books and their due dates.
4. Bus Scheduling System – Maintain bus routes and stop connections.

5. Cafeteria Order Queue – Serve students in the order they arrive.

Student Task:

- For each feature, select the most appropriate data structure from the list below:

- o Stack

- o Queue

- o Priority Queue

- o Linked List

- o Binary Search Tree (BST)

- o Graph

- o Hash Table

- o Deque

- Justify your choice in 2–3 sentences per feature.

- Implement one selected feature as a working Python program with AI-assisted code generation

**Prompt:**

Task: Real-Time Application Challenge – Choose the Right Data Structure

Scenario:

Your college wants to develop a Campus Resource Management System that handles:

1. Student Attendance Tracking – Daily log of students entering/exiting the campus.

2. Event Registration System – Manage participants in events with quick search and removal.

3. Library Book Borrowing – Keep track of available books and their due dates.

4. Bus Scheduling System – Maintain bus routes and stop connections.

5. Cafeteria Order Queue – Serve students in the order they arrive.

Student Task:

- For each feature, select the most appropriate data structure from this list:

- Stack

- Queue

- Priority Queue

- Linked List

- Binary Search Tree (BST)
- Graph
- Hash Table
- Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program.
- Provide a short 4-line observation of the program's behavior.

Expected Output:

1. A clear mapping of each feature → data structure → justification.
2. A Python program implementing one feature (e.g., Cafeteria Order Queue using a Queue).
3. A sample run/output.
4. A concise 4-line observation explaining correctness.

**Code Generated:**

```

11.1.task10.py > ...
1 from collections import deque
2
3 class CafeteriaQueue:
4 def __init__(self):
5 self.queue = deque()
6
7 def place_order(self, student_name):
8 """Add student order to the queue."""
9 self.queue.append(student_name)
10 print(f"Order placed by {student_name}.")
11
12 def serve_order(self):
13 """Serve the next student in queue."""
14 if self.is_empty():
15 print("No orders to serve.")
16 return None
17 student = self.queue.popleft()
18 print(f"Order served for {student}.")
19 return student
20
21 def next_order(self):
22 """Peek at the next order without removing."""
23 if self.is_empty():
24 print("No pending orders.")
25 return None
26 return self.queue[0]
27
28 def is_empty(self):
29 return len(self.queue) == 0
30
31
32 # Example usage
33 if __name__ == "__main__":
34 cafeteria = CafeteriaQueue()
35 cafeteria.place_order("Alice")
36 cafeteria.place_order("Bob")
37 cafeteria.place_order("Charlie")
38
39 print("Next order:", cafeteria.next_order())
40 cafeteria.serve_order()
41 cafeteria.serve_order()
42 cafeteria.serve_order()
43 cafeteria.serve_order() # No orders left
44

```

## Output:

```

[Running] python -u "c:\Users\akshi\OneDrive\Desktop\AI_Assisted_Coding\11.1.task10.py"
Order placed by Alice.
Order placed by Bob.
Order placed by Charlie.
Next order: Alice
Order served for Alice.
Order served for Bob.
Order served for Charlie.
No orders to serve.

[Done] exited with code=0 in 0.137 seconds

```

## Observation:

1. The **Cafeteria Queue** uses Python's deque for efficient FIFO operations.
2. `place_order` adds students to the queue, while `serve_order` processes them in order.
3. `next_order` allows checking the upcoming order without removing it.
4. Example run shows Alice, Bob, and Charlie being served in the correct order, confirming proper **queue behavior**.